

Learning Decentralized Multi-Robot PointGoal Navigation

Takieddine Soualhi , Nathan Crombez , Yassine Ruichek , Alexandre Lombard , and Stéphane Galland 

Abstract—Integrating robots into real-world applications requires effective consideration of various agents, including other robots. Multi-agent reinforcement learning (MARL) is an established field that addresses multi-agent systems problems by leveraging reinforcement learning techniques. Despite its potential, the study of multi-robot systems, particularly in vision-based robotics, remains in its early stages. In this context, this article tackles the PointGoal navigation problem for multi-robot systems, extending the traditional single agent focus to a multi-agent context. To this end, we introduce a training environment designed to address vision-based multi-robot challenges. In addition, we propose a method based on the centralized training-decentralized execution paradigm within MARL to explore three PointGoal navigation scenarios: the SpecificGoal scenario, where each agent has a distinct target; the CommonGoal scenario, where all agents share the same target; and the Ad-hoCoop scenario, which requires agents to adapt to varying team sizes. Our results contribute to lay the groundwork for adopting MARL approaches to address vision-based tasks for multi-robot systems.

Index Terms—Multi-robot systems, vision-based navigation, reinforcement learning.

I. INTRODUCTION

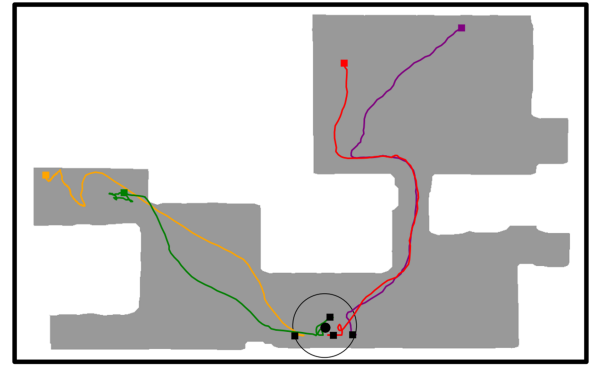
THE world is inherently complex, with numerous entities interacting and cooperating to achieve complex objectives. In real-world settings, the deployment of robots requires effective cooperation among multiple robots. This requires the developed robotics solutions to account for the presence of diverse agents in their environments, ensuring seamless integration and functionality in dynamic settings. These methods must be robust and adaptable, enabling robots to understand and predict the actions of other agents, communicate effectively toward shared or individual goals, and construct and adhere to social norms [1]. By addressing these challenges, robots can be more successfully integrated into various domains, such as transportation [2], industrial automation [3] and logistics [4], enhancing their overall effectiveness and reliability in real-world applications.

Multi-agent reinforcement learning (MARL) is an established field that aims to tackle multi-agent systems problems by leveraging advances in the field of reinforcement learning (RL) [1].

Received 21 October 2024; accepted 14 February 2025. Date of publication 12 March 2025; date of current version 19 March 2025. This article was recommended for publication by Associate Editor C. premachandra and Editor P. Vasseur upon evaluation of the reviewers' comments. (Corresponding author: Takieddine Soualhi.)

The authors are with the CIAD UR 7533, Belfort Montbeliard University of Technology, UTBM, F-90010 Belfort, France (e-mail: takisoualhi@gmail.com). The code for the approach and the training environment are available at: <https://github.com/TakieddineSOUALHI/MARLPointNAV>

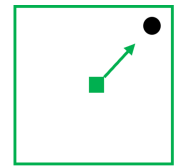
Digital Object Identifier 10.1109/LRA.2025.3550798



(a)



(b)



(c)

Fig. 1. Illustration of the multi-robot PointGoal navigation task. (a) A team of four robots in the CommonGoal scenario. (b) Each robot uses its vision sensor observations and (c) its relative position to target coordinates. The dark circle and boxes represent the success domain and convergence positions.

MARL enables multiple agents to learn and adapt their strategies based on interactions with each other and their environment, showcasing significant potential in real-world applications such as search and rescue [5] and area coverage [6]. Although most existing works operate in grid-world-like environments, relying on low-dimensional observations, the adoption of MARL to solve vision-based robotics problems has recently gained more interest [7], [8]. Nevertheless, addressing vision-based multi-robot problems using MARL is still in its infancy, mainly due to the lack of existing training environments that can provide a high frame rate along with physical realism for realistic simulation of complex tasks.

In this article, the problem of PointGoal navigation for multi-robot systems is addressed, where a team of robots learns to reach a target position. While previous work has focused solely on single agent scenarios [9], [10], [11], we extend this task to a multi-agent context. We argue that studying this problem is fundamental for tackling multi-robot tasks using MARL and further lays the groundwork for solving more complex tasks, such as ImageGoal [12] and ObjectGoal navigation [13]. To address this problem, we first propose a training environment that extends the iGibson 2.0 framework [14] to multi-agent settings. Then,

we propose an approach based on MARL algorithms that adopt a centralized training-decentralized execution (CTDE) paradigm. Specifically, we examine three distinct PointGoal scenarios: the SpecificGoal scenario, the CommonGoal scenario, and the Ad-hoCoop scenario. In the CommonGoal scenario, agents are assigned the same PointGoal, while in the SpecificGoal scenario, agents are given different goals (Fig. 1). The Ad-hoCoop scenario builds on the CommonGoal scenario by assigning a shared goal and introducing changes in team configuration, requiring agents to adapt to different team sizes during deployment. By studying the PointGoal navigation task, we aim to advance the understanding of adopting MARL to solve vision-based multi-robot systems problems, providing groundwork to address more complex tasks. In a nutshell, our contributions can be summarized as follows:

- A large-scale, fast training environment for learning multi-robot vision-based control is provided.
- A MARL method is adopted to address the PointGoal navigation problem in multi-robot systems.
- Large-scale experiments are conducted, and insights are provided for addressing multi-robot visual tasks using MARL.

The remainder of this paper is organized as follows. Section II presents the related work to this paper. Section III outlines the setup of our multi-robot PointGoal navigation task, provides a formulation of the problem within MARL, and presents the proposed MARL approach. Section IV describes the experimental setup and training details. Section V presents the simulation results, comparisons and discusses limitations and insights in training multi-robot systems. Lastly, Section VI presents the conclusions and future work.

II. RELATED WORK

This section presents the related work relevant to this paper, with a focus on learning visual tasks for both single robot and multi-robot systems.

Learning single robot visual navigation: With the emergence of embodied AI, end-to-end approaches using deep reinforcement learning (DRL) have become established in robot navigation, enabling several novel tasks that were not achievable with classical approaches [15]. The PointGoal navigation task is defined as a task in which a robot must navigate to given target coordinates in an unseen static environment using its vision sensor. Numerous works have addressed this problem in a single robot setting. Wijmans et al. [9] demonstrated that this problem can be solved by massively scaling the training on a cluster of GPUs and over 2.5 billion steps of experience (the equivalent of 80 years of human experience). Grande et al. [16] explored PointGoal navigation under sensor and actuator noise without ground-truth localization, introducing auxiliary tasks for learning localization. Sadek et al. [11] extended this by focusing on transferring the PointGoal task to real robots. Building on PointGoal navigation, several spin-off tasks have emerged, such as ImageGoal navigation, where a robot navigates to match a given target image [12], and ObjectGoal navigation, which involves a robot following target language instructions to locate specific objects [13]. Other tasks include interactive navigation [17], where the robot interacts with its environment to successfully navigate and complete its goals.

Learning multi-robot visual tasks: Addressing multi-robot visual tasks using MARL has gained attention in recent embodied

AI research, encompassing several novel multi-agent tasks. Tan et al. [7] introduced multi-agent interactive question answering, where agents cooperate in dynamic environments using a shared structural memory and scene encoder, outperforming single agent methods. This was extended to handle ambiguous human instructions using external knowledge and visual perception for task allocation [18]. Jain et al. [19] addressed multi-agent cooperation tasks, such as moving furniture, using imitation learning, communication, synchronized policies, and coordination loss. A benchmark for multi-agent tidying-up tasks was proposed, where agents collaboratively place misplaced objects using hierarchical decision models [8]. Multi-agent visual semantic navigation, where robots locate target objects using visual observations and class labels, has been addressed by Liu et al. [20], who proposed a hierarchical decision-making framework that includes semantic mapping and learned communication. Wang et al. [21] tackled ImageGoal navigation for multi-robot systems, proposing CollaVN, a multi-agent ImageGoal dataset, and a memory-augmented communication model with private external memory for each agent. Multi-robot visual exploration was recently addressed by Yu et al. [22] and Yang et al. [23], respectively. Yu et al. proposed a MARL framework for asynchronous control of multi-robot systems, while Yang et al. tackled the multi-agent neural SLAM problem, where a team of agents learns policies to collectively build a semantic map online. The application of offline RL and LLMs has been explored by Morad et al. [24], where multi-robot systems learn to follow natural language instructions, with policies trained on minimal data and deployed without fine-tuning.

Discussion: While the proposed approaches show promising results, addressing multi-robot challenges with MARL remains in its early stages. To the best of our knowledge, the PointGoal navigation task for multi-robot systems has not been explored. Most existing works use grid-world-like environments with atomic actions, and often suffer from low rendering speeds, posing a significant bottleneck for training MARL algorithms on complex visual tasks. Wang et al. [21] proposed the CollaVN dataset and adapted GibsonV1 [14] for multi-robot use cases. However, the adapted training environment operates at a low speed. Furthermore, while their work presents interesting concepts, it would benefit from a deeper analysis of key MARL aspects, and the code has not been open-sourced.

III. LEARNING DECENTRALIZED MULTI-ROBOT POINTGOAL NAVIGATION

This section provides an overview of the problem formulation and introduces the proposed approach.

A. Problem Statement

Consider a team of N robots initialized at random starting poses in an unseen scene, tasked with navigating to a target position specified relative to each robot's position. No map is available and each robot must navigate using only its vision sensor and the goal position provided relative to the robot's current position. In addition to reasoning about the scene structure (i.e., obstacles, identifying navigable space), each robot has to learn to adapt to the policies of other agents. Let $g_{i,t}$ be the specified goal for i -th robot at instant t . For our multi-robot PointGoal navigation task, we consider three different scenarios.

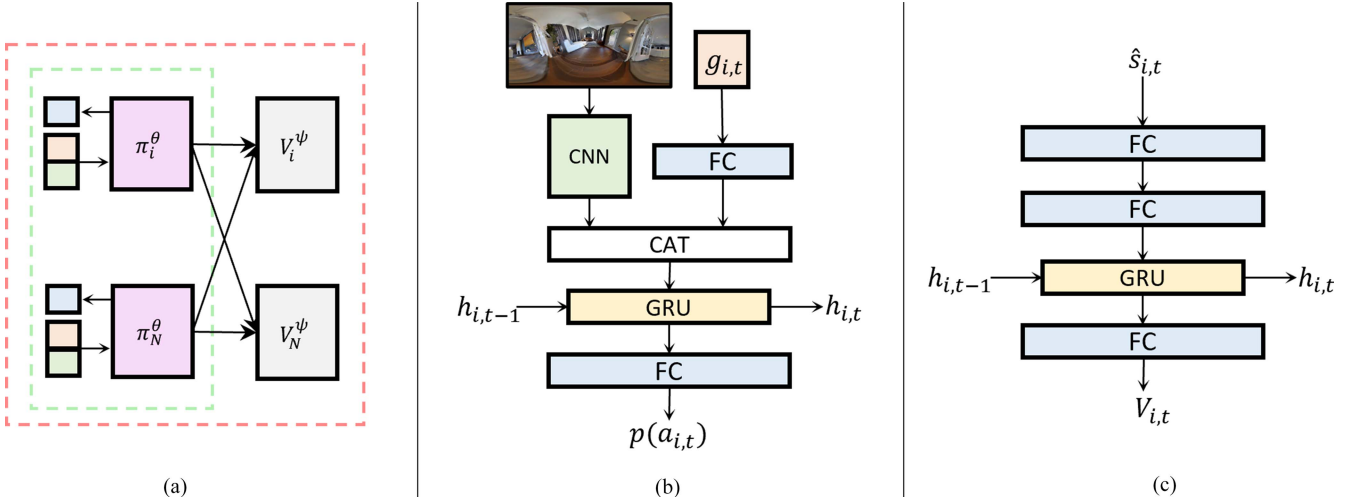


Fig. 2. The proposed multi-robot PointGoal task method: (a) Illustration of the CTDE framework. The dashed red line represents the training phase, whereas the dashed green line represents the test/deployment phase; (b) Illustration of the navigation policy model; (c) The centralized critic model.

CommonGoal: N robots have a common goal, i.e., $g_{1,t} = g_{2,t} = \dots = g_{N,t}$. In this scenario, the robots need to navigate to the target goal within an episode T . This scenario aims to investigate how several agents could align towards reaching a common goal.

SpecificGoal: In this scenario, N robots are each assigned different goal positions, i.e., $g_{1,t} \neq g_{2,t} \neq \dots \neq g_{N,t}$. This setting is particularly interesting for understanding how agents manage to coordinate in when their goals differ.

Ad-hoCoop: In the CommonGoal and SpecificGoal settings, the team size remains fixed during both the training and test phases. In the Ad-hoCoop scenario, the team size is different during training and testing, requiring agents to adapt to different team sizes. This scenario aims at investigating the agent's capacity to generalize and adapt to new team configurations. We note that in Ad-hoCoop, agents are assigned the same goal, similar to the CommonGoal scenario.

B. Problem Formulation

For a system of N agents operating in a shared environment, multi-robot PointGoal task can be formulated as a partially observable stochastic game (POSG). Formally, a POSG is defined by the tuple $\langle N, S, \{A_i\}_{i=1}^N, \{O_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N \rangle$. S represents the finite set of states that describe the environment. In this formulation, each agent i has a finite set of actions A_i and a finite set of observations O_i . The transition model $T: S \times A_1 \times A_2 \times \dots \times A_N \rightarrow [0, 1]$ defines the probability distribution over the next state given the current state and the joint actions taken by all agents. The reward function $R_i: S \times A_1 \times A_2 \times \dots \times A_N \rightarrow \mathbb{R}$ assigns a reward to the i -th agent based on the current state and the joint action of all agents. Given a timestep t , each agent i receives a local observation $o_{i,t} \in O_i$ and executes an action $a_{i,t} \in A_i$ according to a stochastic policy $\pi_i^\theta: O_i \rightarrow A_i$, parameterized by θ . The choice of the stochastic policy here is to foster exploration during training.

Operating under partial observability and decentralized control requires strategies that account for both local observations and potential coordination with other agents. In this context, the learning objective in a partially observable stochastic game is to

find policies $\pi = (\pi_1, \dots, \pi_N)$ such that:

$$\forall i: \pi_i \in \arg \max_{\pi'_i} \mathbb{E}[R_i | \pi'_i, \pi_{-i}], \quad (1)$$

where $\pi_{-i} = \pi \setminus \{\pi_i\}$. This formulation captures the essence of multi-agent interactions in partially observable environments, providing a structured approach to study and develop strategies among multiple intelligent agents. Another option is to model this problem as a decentralized partially observable Markov decision process (Dec-POMDP), which maintains the POSG formulation, except that all agents receive a shared reward, i.e., $R_1 = R_2 = \dots = R_N$. This shared reward fosters the learning of cooperative behavior among the agents. The choice of using POSG here stems from the finding that a shared reward in the context of a multi-robot PointGoal task results in the 'lazy agent' phenomenon [25]. We compare the performance under these two formulations in Section V-C. To further clarify the POSG structure, we now detail individual observation spaces O_i , action spaces A_i , and reward functions R_i .

Observation space: At each timestep t , the i -th agent receives an observation $o_{i,t}$ that consists of an egocentric panoramic image as well as the goal position $g_{i,t}$ relative to the robot's current position.

Action space: We adopt a continuous control scheme to train our agents. At each time step t , the i -th agent executes an action $a_{i,t}$ on the continuous control of linear and angular velocities, expressed in the robot frame.

Reward function: At each timestep t , the i -th robot receives a reward $r_{i,t} = \Delta d_{i,t} + \mathbb{1}_{success} \cdot K + C$, where $d_{i,t}$ is the geodesic distance to the target and $\Delta d_{i,t}$ is the change in distance after taking an action. $\mathbb{1}_{success}$ is a binary indicator that equals 1 for a successful episode, with $K = 10$ as the success reward. $C = -0.05$ is the penalty for collisions with the environment or other agents.

C. Method

We propose an approach that employs a recurrent policy in an actor-critic style, trained using the multi-agent proximal policy optimization (MAPPO) algorithm, which is an extension of

the PPO algorithm [26] for multi-agent settings and operates under a CTDE paradigm (Fig. 2(a)). This choice is motivated by two primary reasons. First, the efficiency and effectiveness demonstrated by MAPPO in addressing common MARL benchmarks. Studies such as Yu et al. [27] and Papoudakis et al. [28] highlight MAPPO's superior performance and efficiency compared to other methods. Second, PPO, the foundation of MAPPO, has been highly successful in addressing single robot complex vision-based navigation tasks, such as ImageGoal navigation [12] and interactive navigation [17].

In a multi-agent setting, each agent i has a navigation policy model π_i^θ that maps observations to actions, and a centralized critic model V_i^ψ that estimates the value function by leveraging information from all agents in the environment. In the following, we detail both models.

1) *Navigation Policy*: Each agent relies on its observation as input to the navigation policy model, which computes the probability distribution for all possible actions: $\pi_i^\theta(a_{i,t}|o_{i,t}, g_{i,t})$. The policy model shown in Fig. 2(b), is a recurrent policy composed of a convolutional neural network (CNN) encoder, which encodes the panoramic observations, and a linear layer, which encodes the goal coordinates.

First, the policy encodes each observation separately. The panoramic observation is passed through the CNN encoder to generate a representation of visual features $v_{o_{i,t}}$, and the goal position $g_{i,t}$ is passed through a fully connected layer to produce a representation $v_{g_{i,t}}$. The outputs of both layers are then concatenated (CAT) to form a joint representation. The vector $v_{\text{joint}_{i,t}}$ resulting from the concatenation is then passed into a 1-layer gated recurrent unit (GRU). Finally, the contextual representation $h_{i,t}$ resulting from the GRU layer is passed through a fully connected (FC) layer to output an action a probability distribution p over actions $a_{i,t}$ as follows:

$$\begin{aligned} v_{\text{joint}_{i,t}} &= \text{CAT} \left(v_{\text{obs}_{i,t}}, v_{\text{goal}_{i,t}} \right), \\ h_{i,t} &= \text{GRU} \left(v_{\text{joint}_{i,t}}, h_{i,t-1} \right), \\ p(a_{i,t}) &= \text{FC}(h_{i,t}). \end{aligned} \quad (2)$$

Each agent optimizes the policy model, using a PPO loss. PPO is a policy gradient DRL algorithm that alternates between sampling a set of trajectories and optimizing a surrogate loss function. The loss of navigation policy is given by:

$$\mathcal{L}_i(\theta_i) = \mathbb{E} [\min(r_t(\theta_i)A_{i,t}, \text{clip}(r_t(\theta_i), 1 - \epsilon, 1 + \epsilon)A_{i,t})]. \quad (3)$$

where $r_{i,t}(\theta_i) = \frac{\pi_i^\theta(a_{i,t}|o_{i,t})}{\pi_i^{\text{old}}(a_{i,t}|o_{i,t})}$ is the probability ratio between old and current policies, $A_{i,t}$ is the advantage function, and clip is a clipping function with a range ϵ .

2) *Centralized Critic Model*: A recurring problem in MARL is the nonstationarity of the environment due to agents changing their policies. In particular, agents can learn policies that overfit to the behaviors of others, resulting in failure when other agents alter their strategies. One common approach to overcome this problem is to adopt a centralized value function $V_i^\psi = f(o_1, \dots, o_N)$ that takes as input the observations of all agents [29]. As a result, the critic of each agent deals with a stationary environment, and during inference, it only needs to access the local information since only the policy model is used.

However, this approach faces a dimensionality curse when solving visual control tasks. Large-scale training of a population of visual navigation agents becomes computationally expensive as the number of agents increases. This is due to the linear increase in both the number of visual encoders required for the agents and their corresponding input dimensions, which leads to slower training and difficulty in scaling to a large number of agents. To address this issue while maintaining a CTDE paradigm, we define a vector $\hat{s}_{i,t}$ as input to the critic network. This vector contains scalar values that can be obtained from the simulation, reflecting the state of all agents. Let $x_{i,t} = [m_{i,t}, \omega_{i,t}, v_{i,t}, g_{i,t}, b_{i,t}]$ be the vector containing the information of the agent i -th, where $m_{i,t}$ represents the spatial position, $\omega_{i,t}$ is the orientation, $v_{i,t}$ is the angular and linear velocities, and $b_{i,t}$ is the bump sensor output. At each instant t , the input to the i -th agent critic is $\hat{s}_{i,t} = [x_{i,t}, \dots, x_{N,t}]$. This allows the critic network to leverage state information from all agents while using less memory compared to image observations, thereby providing better scalability and faster training.

The centralized critic architecture resembles the navigation policy architecture (Fig. 2(c)), with the encoder substituted by two FC layers. For the loss function, each agent i optimizes its corresponding critic model using the mean squared error loss (MSE). This loss is expressed as follows:

$$\mathcal{L}_i(\psi_i) = \mathbb{E} \left[\left(y_{i,t} - V_i^\psi(\hat{s}_{i,t}) \right)^2 \right]. \quad (4)$$

where $y_{i,t} = \sum_{j=t}^{T-1} \gamma^{j-t} r_j + \gamma^{T-t} V_i^\psi(\hat{s}_{i,T})$ is bootstrapped from the last vector $\hat{s}_{i,T}$ and T is the duration of the episode. $\gamma \in [0, 1]$ here is the discount factor

IV. EXPERIMENTAL SETUP

This section provides detailed descriptions of the training dataset we used and the evaluation setup.

A. Dataset

We use the CollaVN dataset [21], derived from the Gibson indoor scenes dataset and generated following the protocol of Chaplot et al. [30]. It includes 35 fixed scenes: 25 for training, 5 for validation, and 5 for testing. Each scenario (CommonGoal, SpecificGoal, Ad-hoCoop) contains 2 M/60 K/120 K episodes for training/validation/test. Navigation episodes are categorized into three difficulty levels based on start-to-goal distance: easy (1.5–3 m), medium (3–5 m) and hard (5–10 m). The data split is detailed below for each scenario:

CommonGoal scenario: This task contains three subsets for different team sizes (that is, $N = \{2, 3, 4\}$), where agents in each episode are assigned the same target position. Each subset contains 10K episodes per training scene and 0.5K/1K episodes per validation/test scene per difficulty.

SpecificGoal scenario: Similar to the previous scenario, this scenario also contains three subsets for different team sizes, where agents in each episode are assigned different target positions.

Ad-hoCoop scenario: This scenario includes two subsets for different team size configurations ($N : 2 \rightarrow 3$ and $N : 3 \rightarrow 2$), with agents in each episode assigned the same target position. Like previous scenarios, each subset contains 10K episodes per training scene and 0.5K/1K episodes per validation/test scene

TABLE I
COMPARISON OF TRAINING ENVIRONMENTS

Environments	Frame rate (FPS) $\uparrow$	Memory (%) $\downarrow$	100K steps (hours) $\downarrow$
Wang et al. [21]	13	100	2.14
Ours	62	54	0.49

The upper arrow indicates improvement with an increase, while the lower arrow indicates improvement with a decrease.

per difficulty. The training set is identical to the CommonGoal scenario.

B. Metrics

To evaluate our multi-robot PointGoal navigation task, we use the success rate (SR) and success weighted by path length (SPL) metrics [15], described below:

SR: An episode is considered successful for an agent if it reaches the target within a radius of 1 m. Let $S_{k,n} \in \{0, 1\}$ be a binary operator of success in episode k for agent n . The SR for N robots over K evaluation episodes is given by:

$$SR = \frac{1}{KN} \sum_k \sum_n S_{k,n}. \quad (5)$$

SPL: Let $l_{k,n}^g$ be the geodesic distance from the start position to the goal of agent n in episode k , and $l_{k,n}$ the length of the actual trajectory of agent n in episode k . The SPL for N robots across K evaluation episodes is given by:

$$SPL = \frac{1}{KN} \sum_k \sum_n S_{k,n} \cdot \frac{l_{k,n}^g}{\max(l_{k,n}^g, l_{k,n})}. \quad (6)$$

V. EXPERIMENTAL RESULTS

This section provides a performance analysis of the proposed training environment, evaluation results across earlier scenarios, and insights into addressing multi-robot visual tasks. Results are evaluated over three random seeds.

A. Comparison Between Training Environments

Training vision-based multi-robot systems requires environments that balance physical realism and rendering speed. We propose a simulation environment that extends iGibson 2.0 [14] for multi-robot configurations. This environment relies on the PyBullet simulator for physics simulation and neural image rendering, enabling faster training of robots for deployment in real-world scenarios. Our adaptation supports loading and training multiple robots simultaneously and provides data wrappers to efficiently manage diverse observations from a team of robots. In comparison, Habitat [9], commonly used for single robot PointGoal navigation, does not support multi-robot simulation. Our environment enables studying more complex problems like multi-robot mobile manipulation and social navigation. We compare the performance of our training environment with that of Wang et al. [21]. In this experiment, two robots are trained in 20 parallel environments with panoramic observations. We conduct the experiments to measure the training efficiency in terms of frames per second (FPS), time to reach 100 K steps (in hours), and GPU memory usage, as reported in Table I. We use two NVIDIA A6000 GPUs with 48 GB of memory each.

Our environment operates at 62 FPS, 4.79 times faster than Wang et al. [21], which allows for 100 K steps to be completed

in 0.5 hours. This efficiency stems from an optimized software pipeline that eliminates bottlenecks and excessive memory usage, with both policy optimization and observation rendering running entirely on the GPU, avoiding alternation between the CPU and GPU. It is important to note that the scalability of our environment varies with the number of robots and the type of vision sensor used. With a panoramic vision sensor, the FPS drops from 62 to 35 and then to 23 as the number of robots increases from 2 to 3 and 4, respectively. In contrast, a perspective RGB sensor provides higher FPS, with 145 for 2 robots, 70 for 3 robots, and 50 for 4 robots.

B. Results and Comparative Analysis

We compare the following methods in the multi-robot PointGoal navigation task:

Random policy: The simplest heuristic in RL involves an agent taking random actions to reach the goal coordinates.

Single visual navigation robot (SVNR): A single agent is trained on the PointGoal task, and multiple copies are deployed. SVNR highlights the impact of multi-agent learning.

Multi-Robot visual navigation (MRVN): Multiple robots are trained and then deployed according to a specific scenario.

Fig. 3 shows sample trajectories for varying robot configurations on hard episodes from the Uvalda scene across different scenarios. Under MRVN, the robots successfully navigate to the goal coordinates in complex scenes, dynamically adjusting their paths to avoid collisions. These results suggest that using MARL, robots can learn implicit coordination directly from pixels and reward signals, without decentralized communication. In contrast, SVNR performs worse than MRVN, as expected due to the lack of a multi-agent setting during training.

Table II highlights the evaluation results obtained on the SpecificGoal scenario. For MRVN, our results show that performance drops as the difficulty level of the episode increases, which is intuitive since the distance the robots have to travel increases with difficulty. Furthermore, we find that performance is maintained as the number of robots increases. This result suggests that performance is invariant to scaling the multi-robot system. Similarly, the same conclusions can be drawn for the CommonGoal scenario (Table III). The invariance in performance across different goals can be attributed to the reward function setting, as in each scenario, agents receive a reward that corresponds to their own goal and a penalty for collisions with other agents and the environment.

The remarks previously made on MRVN still hold for SVNR on the SpecificGoal and CommonGoal scenarios with the performance dropping as the difficulty increases. However, performance also drops as the number of robots increases. We argue that this is because, as the number of robots increases, each robot struggles to adapt to the policies of other robots due to their absence during training.

Table IV presents the evaluation results for the Ad-hoCoop scenario, which assesses the generalization capabilities of MRVN to previously unseen agents. The results show that the system maintains performance when the number of agents decreases ($3 \rightarrow 2$), but there is a slight drop when the number of agents increases during inference ($2 \rightarrow 3$). This drop in performance arises from MARL methods struggling to generalize strategies for handling unseen agents, as highlighted by Mirsky et al. [31]. Addressing this challenge involves exploring research directions such as enhancing agent modeling with auxiliary tasks

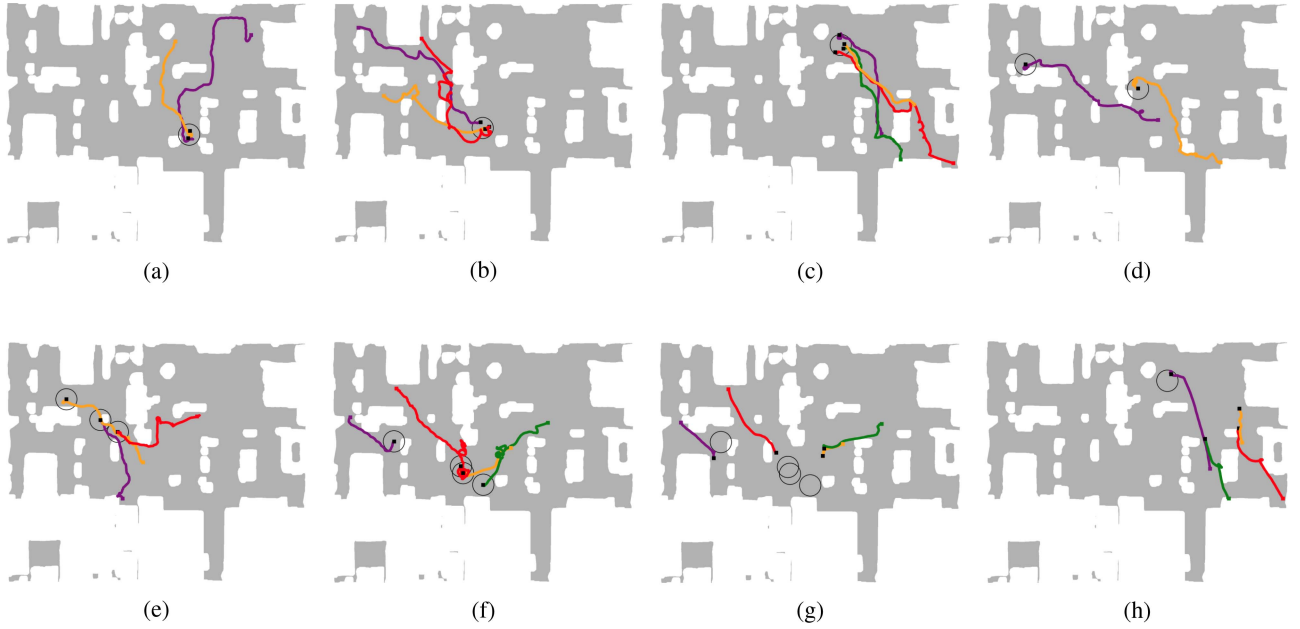


Fig. 3. Sample trajectories for various robot settings on the Hard test set from the Uvalde scene for MRVN on (a)–(c) CommonGoal scenario and (d)–(f) SpecificGoal scenario. (g)–(h) Trajectories performed by SVN compared to MRVN.

TABLE II
PERFORMANCE ON THE SPECIFICGOAL SCENARIO FOR DIFFERENT TEAM SIZES

N	Method	Easy (1.5-3 m)		Medium (3-5 m)		Hard (5-10 m)		Average	
		SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$
1	Random Policy	9.35	0.04	0.10	0.00	0.00	0.00	3.15	0.02
2	SVNR	81.40	0.80	51.16	0.50	11.82	0.09	48.46	0.47
	MRVN	88.10	0.85	61.66	0.58	18.9	0.13	56.22	0.52
3	SVNR	78.57	0.76	48.74	0.42	11.96	0.09	46.43	0.44
	MRVN	85.13	0.82	56.26	0.53	14.81	0.11	52.86	0.49
4	SVNR	77.44	0.75	45.03	0.40	0.05	0.04	40.84	0.39
	MRVN	85.81	0.82	55.86	0.53	14.42	0.11	52.04	0.49

TABLE III
PERFORMANCE ON THE COMMONGOAL SCENARIO FOR DIFFERENT TEAM SIZES

N	Method	Easy (1.5-3 m)		Medium (3-5 m)		Hard (5-10 m)		Average	
		SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$
2	SVNR	81.26	0.80	49.86	0.48	12.30	0.09	47.81	0.47
	MRVN	84.02	0.82	58.06	0.51	19.46	0.16	53.85	0.49
3	SVNR	80.34	0.79	51.93	0.49	12.16	0.09	48.15	0.46
	MRVN	85.15	0.82	56.90	0.54	14.33	0.12	52.13	0.50
4	SVNR	79.57	0.76	48.7	0.46	0.06	0.04	42.78	0.42
	MRVN	85.37	0.83	56.18	0.53	14.12	0.12	53.85	0.49

or pretrained visual representations to infer generalizable teammate attributes, as well as training with diverse agent populations to expose MARL to a wide range of strategies and configurations. This raises important questions about how to improve visual representations in MARL to enable better generalization to unseen agents.

C. Ablations and Design Choices

We further study the impact of vision sensor, critic configuration, and reward setting on learning vision-based navigation for

multi-robot systems. We report the average performance across the test set (easy, mid, hard).

Critic setup: In this experiment, we study the impact of the critic's input for two robots on the SpecificGoal scenario, comparing three critic configurations: independent critic (i.e., each agent's critic is conditioned on its local observations), centralized critic - observations (i.e., each agent's critic is conditioned on the observations of all agents in the environment), and centralized critic - state (i.e., each agent i is conditioned on the the previously defined $\hat{s}_{i,t}$).

The results in Table V show that using an independent critic leads to a decrease in performance, due to the nonstationarity

TABLE IV
PERFORMANCE ON THE AD-HOCOOP SCENARIO FOR DIFFERENT TEAM SIZES

N	Method	Easy (1.5-3 m)		Medium (3-5 m)		Hard (5-10 m)		Average	
		SR ↑	SPL ↑	SR ↑	SPL ↑	SR ↑	SPL ↑	SR ↑	SPL ↑
3 → 2	MRVN	84.73	0.82	57.24	0.55	14.81	0.14	52.26	0.50
2 → 3	MRVN	84.22	0.82	53.06	0.47	13.34	0.12	50.21	0.48

TABLE V
COMPARISON OF CRITIC INPUT CONFIGURATIONS

Critic setting	Average	
	SR ↑	SPL ↑
Independent critic	52.15	0.48
Centralized - Observations	56.55	0.52
Centralized - State	56.22	0.52

TABLE VI
COMPARISON OF VISION SENSORS

Vision Sensor	Average	
	SR ↑	SPL ↑
Perspective - RGB	48.84	0.41
Perspective - RGBD	51.61	0.44
Panoramic - RGB	52.15	0.48

of the value function, as the agent lacks access to the global state of the environment. On the other hand, the Centralized - Observations configuration yields better performance, as it alleviates the nonstationarity problem. However, as discussed earlier, for visual control tasks, using observations from all agents as input to the critic introduces practical limitations, particularly computational ones, since the input size of the critic and the number of required visual encoders increase linearly. In contrast, using a Centralized - State configuration that relies on computationally efficient information from the simulator retains the advantages of the Centralized - Observations configuration in addressing the nonstationarity problem, while avoiding the computational bottleneck.

Vision sensor comparison: In this experiment, we analyze performance under an independent critic and three different vision sensors: RGB, RGBD perspective, and RGB panoramic sensors. The results are reported in Table VI.

The Perspective RGBD sensor is commonly used in PointGoal navigation due to its depth channel, which offers better performance than simple RGB sensors. This advantage extends to multi-robot settings, as shown in Table VI. However, the panoramic vision sensor outperforms both RGB and RGBD perspective sensors. We argue that its wider field of view provides better perception of the environment and other agents.

Reward setting: In this experiment, we analyze the performance of three robots in the CommonGoal scenario under two different reward settings: an independent reward setting, following a POSG formulation, and a shared reward setting, following a Dec-POMDP formulation. The results are reported in Table VII.

Under a shared reward, we found that only one of the three agents successfully learned to reach the desired pose, while the others learned to remain stationary, resulting in a significant performance drop. This is due to the multi-agent credit assignment problem, where each agent struggles to infer its contribution to

TABLE VII
COMPARISON OF REWARD SETTINGS

Reward setting	Average	
	SR ↑	SPL ↑
Shared reward	19.12	0.17
Independent reward	52.13	0.50

the shared reward. This issue can be addressed through value decomposition methods, which we leave for future work [25].

D. Discussion and Limitations

In this section, we summarize suggestions for training multi-robot vision-based tasks, along with the limitations of our approach and insights for future work.

What training paradigm should be used? CTDE is the paradigm to adopt as it allows for addressing the nonstationarity problem in MARL by leveraging actor-critic methods. For the critic input, we recommend using inputs from the simulation other than image observations to avoid computational bottlenecks.

Which vision sensor should be used for training my multi-robot vision task? We suggest using a panoramic vision sensor as it allows for better perception of other agents and the scene.

Which reward setting should be used: shared or individual rewards? The choice of reward setting depends on the task. For explicitly cooperative tasks, a shared reward fosters cooperative behavior, while independent rewards may lead to competitive or egoistic agents. In tasks like PointGoal navigation, individual reward functions are straightforward to design and do not induce competitive behavior. In contrast, for tasks like cooperative multi-robot manipulation, it is more difficult to design individual rewards that encourage cooperative policies, making a shared reward more practical. Nonetheless, using a shared reward can result in lazy agents, as they may rely on others to achieve the goal.

Our approach has several limitations, such as relying on access to perfect relative target positions during training and assuming noiseless actuation. We also observe general inefficiencies in training MARL algorithms. These challenges include difficulties in scaling to larger teams, particularly due to the memory overhead involved in neural network updates. Additionally, large-scale parallelization is often asynchronous, with centralized gradient computation occurring only after rollouts are collected, which significantly limits the training speed. Moreover, agents operate synchronously, meaning all agents act and receive observations at the same time. These issues can be mitigated by employing distributed decentralized training for RL agents [9] and asynchronous action execution for MARL agents [22]. While we did not study real-world transfer in this work, Sadek et al. [11] investigated transferring the PointGoal navigation policies from simulation to real robots. They found that fine-tuning models on

target scenes achieved competitive performance, whereas zero-shot transfer resulted in lower performance. However, this study addressed only single agent setups, leaving multi-agent strategy transfer an open question. We note that domain randomization could enable zero-shot transfer by improving model robustness.

VI. CONCLUSION

This article studies the problem of learning multi-robot Point-Goal navigation. To address this problem, we build a training environment that provides for a high frame rate and large-scale simulation of multi-robot systems. We propose an approach based on the centralized training - decentralized execution paradigm. The results provide a template for training and understanding vision-based multi-robot tasks using multi-agent reinforcement learning. Future work will focus on addressing the limitations of our approach and expanding beyond the PointGoal task to tackle more complex challenges, such as ImageGoal and ObjectGoal navigation, multi-robot mobile manipulation, and real-world transfer. Additionally, we plan to incorporate value decomposition methods and inter-agent communication to enhance the learning of cooperative strategies among robots.

REFERENCES

- [1] A. Dafoe, Y. Bachrach, G. Hadfield, E. Horvitz, K. Larson, and T. Graepel, "Cooperative AI: Machines must learn to find common ground," *Nature*, vol. 593, no. 7857, pp. 33–36, 2021.
- [2] J. Dinneweth, A. Boubezoul, R. Mandiau, and S. Espié, "Multi-agent reinforcement learning for autonomous vehicles: A survey," *Auton. Intell. Syst.*, vol. 2, 2022, Art. no. 27.
- [3] Y. Shen, B. McClosky, J. W. Durham, and M. M. Zavlanos, "Multi-agent reinforcement learning for resource allocation in large-scale robotic warehouse sortation centers," in *Proc. 62nd IEEE Conf. Decis. Control*, 2023, pp. 7137–7143.
- [4] X. Li, J. Zhang, J. Bian, Y. Tong, and T.-Y. Liu, "A cooperative multi-agent reinforcement learning framework for resource balancing in complex logistics network," in *Proc. 18th Int. Conf. Auton. Agents MultiAgent Syst.*, 2019, pp. 980–988.
- [5] J. Parker, E. Nunes, J. Godoy, and M. Gini, "Exploiting spatial locality and heterogeneity of agents for search and rescue teamwork," *J. Field Robot.*, vol. 33, no. 7, pp. 877–900, 2016.
- [6] A. A. Adepegba, S. Miah, and D. Spinello, "Multi-agent area coverage control using reinforcement learning," in *Proc. Int. FLAIRS Conf.*, 2016, pp. 368–373.
- [7] S. Tan, W. Xiang, H. Liu, D. Guo, and F. Sun, "Multi-agent embodied question answering in interactive environments," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 663–678.
- [8] X. Liu, D. Guo, X. Zhang, and H. Liu, "Heterogeneous embodied multi-agent collaboration," *IEEE Robot. Automat. Lett.*, vol. 9, no. 6, pp. 5377–5384, Jun. 2024.
- [9] E. Wijmans et al., "DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–14.
- [10] J. Ye, D. Batra, E. Wijmans, and A. Das, "Auxiliary tasks speed up learning point goal navigation," in *Proc. 2020 Conf. Robot Learn.*, 2021, pp. 498–516.
- [11] A. Sadek, G. Bono, B. Chidlovskii, and C. Wolf, "An in-depth experimental study of sensor usage and visual reasoning of robots navigating in real environments," in *Proc. 2022 Int. Conf. Robot. Automat.*, 2022, pp. 9425–9431.
- [12] G. Bono, L. Antsfeld, B. Chidlovskii, P. Weinzaepfel, and C. Wolf, "End-to-end (instance)-image goal navigation through correspondence as an emergent phenomenon," in *Proc. Int. Conf. Learn. Representations*, 2024.
- [13] S. Fan, R. Liu, W. Wang, and Y. Yang, "Navigation instruction generation with BEV perception and large language models," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 368–387.
- [14] C. Li et al., "iGibson2.0: Object-centric simulation for robot learning of everyday household tasks," in *Proc. 5th Conf. Robot Learn.*, 2022, pp. 455–465.
- [15] P. Anderson et al., "On evaluation of embodied navigation agents," 2018, *arXiv:1807.06757*.
- [16] G. Grande, D. Batra, and E. Wijmans, "Realistic pointgoal navigation via auxiliary losses and information bottleneck," *arXiv preprint*, 2021, *arXiv:2109.08677*.
- [17] W. Li, R. Hong, J. Shen, L. Yuan, and Y. Lu, "Transformer memory for interactive visual navigation in cluttered environments," *IEEE Robot. Automat. Lett.*, vol. 8, no. 3, pp. 1731–1738, Mar. 2023.
- [18] X. Liu, X. Li, D. Guo, S. Tan, H. Liu, and F. Sun, "Embodied multi-agent task planning from ambiguous instruction," *Robot.: Sci. Syst.*, 2022.
- [19] U. Jain, L. Weihs, A. Farhadi, S. Lazebnik, A. Kembhavi, and A. Schwing, "A cordial sync: Going beyond marginal policies for multi-agent embodied tasks," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 471–490.
- [20] X. Liu, D. Guo, H. Liu, and F. Sun, "Multi-agent embodied visual semantic navigation with scene prior knowledge," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 3154–3161, Apr. 2022.
- [21] H. Wang, W. Wang, X. Zhu, J. Dai, and L. Wang, "Collaborative visual navigation," 2021, *arXiv:2107.01151*.
- [22] C. Yu et al., "Asynchronous multi-agent reinforcement learning for efficient real-time multi-robot cooperative exploration," in *Proc. 2023 Int. Conf. Auton. Agents Multiagent Syst.*, 2023, pp. 1107–1115.
- [23] X. Yang et al., "Active neural topological mapping for multi-agent exploration," *IEEE Robot. Automat. Lett.*, vol. 9, no. 1, pp. 303–310, Jan. 2024.
- [24] S. Morad, A. Shankar, J. Blumenkamp, and A. Prorok, "Language-conditioned offline RL for multi-robot navigation," 2024, *arXiv:2407.20164*.
- [25] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. 17th Int. Conf. Auton. Agents MultiAgent Syst.*, 2018, pp. 2085–2087.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017 *arXiv:1707.06347*.
- [27] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 24611–24624.
- [28] G. Papoudakis, F. Christianos, L. Schäfer, and S. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," in *Proc. 1st Int. Workshop Reinforcement Learn. Energy Manage. Buildings & Cities*, 2021, pp. 15–19.
- [29] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, A. Pieter, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6382–6393.
- [30] D. S. Chaplot, R. Salakhutdinov, A. Gupta, and S. Gupta, "Neural topological SLAM for visual navigation," in *Proc. 2020 IEEE/CVF Conf. Computer Vis. Pattern Recognit.*, 2020, pp. 12872–12881.
- [31] R. Mirsky et al., "A survey of ad hoc teamwork research," in *Multi-Agent Syst.*, 2022, pp. 275–293.